

My Horrible Career

“Hilariously relatable”

—Stephen Ocampo



JOHN ARUNDEL
with **ZACK PROSER**

Contents

Praise for “My Horrible Career”	3
About the authors	4
1. How it’s going, vs how it started	5
2. Not a real developer	12
3. Master of my domain	18
4. If you need the money, don’t take the job	23
5. Will write for food	29
About this book	38
What I do	38
Feedback	38
Free updates to future editions	38
Join my Code Club	39
Code For Your Life	39
For the Love of Go	40
Further reading	40
Acknowledgements	41

Praise for “My Horrible Career”

Hilariously relatable reading. I nodded so much at some points I gave myself a neck-ache.

—Stephen Ocampo

I really enjoyed this book. It's great to hear about people's successes, but much more valuable to hear about their failures. John seems to have had plenty of both.

—Cary Witney

AMANDA: *Every time I think it's over, another enemy emerges from Daniel's past, like some twisted game of Whac-A-Mole.*

—“Cobra Kai”

About the authors

Zachary Proser is a full-stack hacker, technical writer and consultant with over a decade of software development experience. He consults with companies around the world.

His technical interests include building large scale systems, open-source development, combining developer tools to achieve workflow zen, infrastructure as code and AI. Zachary is a passionate lifelong learner and believes in sharing knowledge freely and openly, which is why he open-sources the vast majority of his work.

In his free time, Zachary is an avid walker, gamer and maker, and can be found building furniture for his wife Katia and son Zachary. He blogs and shares insights at zack-proser.com.

John Arundel is... well, you'll see.

1. How it's going, vs how it started



How I failed upwards, and finally found a job I can't be fired from

I recently had a fascinating, and wide-ranging, conversation with ace developer advocate Zack Proser, of the highly recommended [Supercharge Your Dev Skills](#) blog. We're both very interested in tech careers and how they work (or don't), and Zack had some penetrating questions about my own career, and whether or not I'm any good at taking my own advice. (Spoiler alert: no.)

In this first part, we'll talk about the beginnings of my career, from unemployed CS bum to tech writer, sysadmin, and "under construction" web designer. Over to Zack:

John, you have a lot to say in your recent book [Code For Your Life](#) about the importance of having a plan for your career, and it really resonated with me. It's made me stop and think a bit about the goals that I always just sort of assumed I'd achieve one day, and ask questions like "When, though?". And "How, exactly?"

I know, right? There's a very funny British zombie movie from the 2000s called [Shaun of the Dead](#), directed by Edgar Wright, in which Simon Pegg plays a guy who's pushing

thirty, but feeling a bit directionless, and drifting through his life.

He works in a hi-fi store, but he's the oldest of the sales assistants, and there's a great scene where he's talking to one of the juniors, played by Rafe Spall:

SHAUN: *Look, Noel, I know you don't want to be here forever. Neither do I. I've got things I want to do with my life.*

NOEL: *When?*

I am Shaun, needless to say. Perhaps there are many other Shauns reading this and smiling ruefully.

Actually, I was even older than Shaun when I first started thinking seriously about what I wanted out of a career, beyond just making rent and having somewhere to go every day. And I think that's probably a pretty typical experience.

Most people **don't really have a career plan**, as such. They're more just sort of winging it. At first, it's astounding to learn that people are willing to pay you to sit in a comfy chair in a warm room, diddling around with computers. I mean, we'd probably do it for free, wouldn't we?

That's true, but I think I'll edit this part out—we don't want everybody to know that.

Fair point. But it's certainly a delightful thing to realise, as a young person entering upon adult life, that you have a talent, or an inclination, that's going to give you a certain amount of employability. We're very lucky that computer jiggy-pokery is economically valuable to society (at least, for now).

However, it's such a broad field that just because you'll probably be making a decent salary after a few years, that doesn't mean you'll necessarily be doing work that's interesting, enjoyable, or valuable. We can probably all think of jobs we've done in the past where that wasn't the case.

And, once you reach a point where you are making enough money to support yourself (and maybe a family), is that it? What's your life really about, when it comes down to it? Are you doing something that keeps you engaged and excited, that makes you happy, that has some meaningful positive effect on the world?

If not, when are you going to start? And how do you get there from here? These are the kind of questions I'm encouraging people to ask themselves, and I hope the book gives them some sort of useful framework to start thinking about it, and maybe filling in some of the answers as time goes on.

The book says “Start at the end”, so let's do that. What is your job now, and how do you feel about it?

When people ask me what I do, and I'm in a hurry, I usually just say “writer”, because people tend to stop listening after that. If I've got more time, though, and they look interested, I might add “and mentor”.

And it's pretty great! You know how you always wish you had a *slightly* better job? Well, I no longer wish that. I can't imagine doing anything more fun or rewarding than what

I'm doing now.

Just explain what you mean by “mentor”, because that’s not usually a job title.

Well, right. I think we’re familiar with the idea of mentoring as being a mixture of teaching, coaching, guidance, and general support. We all do that to some extent, I hope, but in my case I also provide mentoring as a professional service. I have a number of students, or mentees, whatever you want to call them, that I work with regularly as part of the [BIT](#) software engineering program.

I also write [books](#), mostly about Go, but occasionally about other things, such as careers. That’s also part of the mentoring, of course: it turns out a lot of the skills you need to succeed as a software developer are analogue, not digital. In other words, dealing with people, processes, management, and the business side of things.

And you know a bit about that, because you have your own independent business?

That’s right. I’ve run my own company, Bitfield Consulting, for about fifteen years, and over that time I’ve gradually moved from “mostly consulting” to “mostly writing and mentoring”. Maybe I should change the name, but I’ve had the T-shirts printed now.

We’ll get to consulting later on, but tell me a bit about your mentoring.

Well, I love working directly with students, pair programming with them, debugging, talking through design problems, and giving them feedback on their projects. It’s great fun, and I get as much out of it as they do.

For one thing, I really enjoy seeing them blossoming as software engineers, succeeding in their careers, and achieving their dreams. It’s nice to feel that I’ve played some tiny role in that, even though it’s often just a case of giving them confidence in the skills they already have.

You know, sometimes you just need someone to say “Actually, you can do this!”

That really does help, especially coming from someone much more experienced.

I’ve been programming for forty-plus years, I’ve worked in dozens of jobs in multiple areas of tech, consulted for companies all over the world, and published many books, but I *still* feel like an imposter. I mean, there’s so much stuff I don’t know, and so many people who are a heck of a lot more expert than me.

So if *I* feel like that, I know it’s much worse for people who are just starting out. A bit of friendly encouragement and guidance can help, I think.

Something that also features in your books. It often feels like you’re talking directly to the reader, saying “You can do this!”

I hope so. In fact, I couldn’t write the books without having had the mentoring experience.

I mean, you don’t really understand something until you’ve explained it to someone else—ideally, several people. When you see the idea from all those different perspectives, it’s much easier to write about it.

There's a lot of different aspects of software engineering: design, user interfaces, APIs, readability, language syntax, the standard library, the tech stack, and so on. In teaching these things to students, I learn about what people do and don't find intuitive or easy, what puzzles them, what causes confusion, and what helps to resolve it. All of that learning feeds into the book writing, of course.

Sometimes I'll be reading something you've written and a question about it starts to form in my mind, but then in the very next paragraph, you answer that same question!

That's definitely what I'm going for. It's not mind-reading: it's just that I've learned to anticipate what questions people are going to have about a certain subject, and I'll try to answer them in the text.

I hate it when I read something and at the end I'm still puzzled about a bunch of stuff. You want to call up the author and say "What did you *mean* by this, though? Stop talking in riddles!"

People can be very knowledgeable about something, but not necessarily very good at expressing it clearly.

That's very true, and conversely there are some terrific writers who just don't really have much experience of actually *doing* the thing they're writing about.

As for me, I'm not the world's greatest software engineer, and I don't even claim to be a brilliant writer (I'll leave that for others to say). I'm just the overlap in the middle of that Venn diagram.

I know a few things, and I can write about them clearly enough that it at least helps people get the basic idea, so they can jump-start their own learning.

That sounds like a pretty good career, and I can see why you enjoy it. So let's talk about how it all started. What was your own career plan, and when did you first start thinking about it?

This will make me sound like a massive hypocrite, but I never really had a career plan as such. I just bounced around the tech industry for a few decades and ended up somewhere more or less randomly that's turned out to be pretty great.

If you're content to do the same, that's fine by me. Maybe you'll be just as lucky (or privileged) as I was. On the other hand, maybe you won't.

Maybe it's a good idea to be a little smarter than I was, and start thinking about this stuff earlier on in life. That way, you can take control of where you end up, instead of just having it happen to you while you're not looking.

Did you always know what you wanted to be, even if you didn't specifically plan out how to get there?

Not particularly. It's more that I was never really any good at anything else.

I was a [standard computer nerd](#) as a kid, of course, and giving me detention in the computer lab was like throwing Brer Rabbit into the briar patch. I did okay in the traditional CS pathway subjects like math and physics, and indifferently in others.

The choice facing me at university level was really between playing with computers for three years (in a nice warm lab), or something involving hard work and study. So that was a bit of a no-brainer.

What was your first job after college, then?

Well, to my surprise and alarm, it turned out that companies were not queueing up to offer me employment. Most CS graduates look for software development jobs, and so did I, but I didn't have much luck. I applied to a lot of companies, but gradually accumulated a huge pile of rejection letters. You name it, I didn't get it.

In some cases, I think I got the rejection letter before I'd even applied.

You got decruited.

I think the word must have got around somehow. To be fair, while I technically managed to acquire a degree, the actual *marks* I got didn't really bear very close inspection, and I wasn't in any other way an outstanding candidate.

I had a great time at university, though: I learned a great deal about all sorts of things, including life, love, and myself. I just didn't learn a lot about *databases*.

Possibly because you didn't attend any of the lectures?

That's true, but as I explained to the professors at the time, I had better things to do. (So did they, I'm sure.)

So, no regrets, but I wasn't an academic superstar, or even a minor planet of any kind. I was a halfway-decent programmer, at least for a kid fresh out of school, but there were plenty of people who were way smarter and more experienced than me.

The trouble is, they were all applying to the same jobs as I was.

It seemed like everyone else from my cohort got hired, often working for merchant banks at eye-popping starting salaries, while I was still skulking around the Unix lab at odd hours, trying to look like an undergrad who had a right to be there, instead of an unemployed man with an expired student ID, who didn't.

The only *other* thing I had even the slightest aptitude for was writing. I would turn in term papers on compiler design and context-free grammars, or whatever, and they would be written with an inappropriate degree of zip and élan, like something in *Wired* or *The Verge*.

I can just imagine you adding a little zip to your term papers. Élan, even.

I got marked down for that a lot, because you're supposed to write academic papers in a rather stuffy voice, with lots of jargon. The denser the jargon, and the more obscure

the point, the more impressive it sounds and the higher your grade. I failed at that completely, because it seems like such a pointless activity. I want to write technical stuff that's (a) interesting and fun to read, and (b) that people can actually understand.

When?

Don't *you* start. I ended up getting a job as a technical writer for a small computer company in London (actually, it was a fair-sized company, but they made small computers).

I wrote manuals, help files, the little quick-start inserts you get with consumer electronics showing you where to plug things, and any arbitrary bits of text as needed ("Wipe clean periodically with a soft, dry, lint-free cloth. Never submerge the device in water.").

And that's how you became a writer?

Well, yes, but via a number of interesting detours. I learned a lot about the craft of writing in that job: how to write in different registers for different audiences, how to edit my work, how to edit other people's work, how to survive your work *being* edited by other people (it's tough, don't let anyone kid you).

But if you've written eight or ten user manuals, you've more or less written them all, and I was rather restless and dissatisfied. I was surrounded by smart programmers of all kinds: people who understood hardware, people who worked on real, grown-up desktop applications, people who knew about operating systems (they were *writing* operating systems). But, being merely a lowly writer of *words*, I wasn't invited to write any actual code (and rightly so, as I now reflect).

That doesn't sound like a situation that would really suit someone like you.

It chafed, my friend. I was learning a lot by osmosis, and because tech writers have to work pretty closely with the programmers who are building the stuff they're supposed to document. But I couldn't help getting involved in other things, just by overhearing conversations or seeing people doing stuff, and wanting to join in.

It happened that the company had its own NNTP server, which is an ancient protocol that very few people reading this will have heard of. Just as HTTP is the basis of the web, NNTP was the basis of [Usenet](#), which was the predecessor of things like forums, Twitter (or whatever it's called this week), and (though it pains me to say this) Reddit. Anyway, in those pre-web days companies would often host their own "news" (that is, Usenet) servers to save on bandwidth.

I'm going to say you somehow got access to that server.

There was a dusty old PC under a desk somewhere running some vaguely acceptable form of Unix such as Xenix, and it ran the news server. Everyone was very nervous of it, because it was Unix (the opposite of Lex in [Jurassic Park](#), I suppose). So I got involved in running this machine: software updates, backups, the usual care and feeding. Like Twitter, the firehose never shut off, and the machine had a tendency to fill up its disk if you didn't regularly trim some of the binary newsgroups. (Yes, [uencoded porn](#) was a thing. I don't know if anyone browsed it at work.)

Eventually the web came along, which shows you how long ago this was. My little estate expanded to include a web server (Windows NT running IIS, four words I'm happy I'll never hear again). I even worked on the firm's early website, when the main framework for front-end development was Notepad and HTML.

So that's where you get your righteous Web design skills from? Not even being sarcastic, hardly.

My HTML was *tight*. Tiled backgrounds, flashing marquees, “under construction” GIFs, frames, whatever I could jam in there. Flash, JavaScript, and even CSS were a few years away, but I did my best, and my websites were the most “My eyes! The goggles do nothing!” that anyone could possibly wish for. Not much has changed about that, as you can see.

So when did you first manage to get paid for softwaring, instead of just writing and artisanal server crafting?

When I moved on from the PDA company I fell (again rather coincidentally) into a job running Unix web servers, this time for an online news site (which, like most online news sites, is now long defunct).

I also did some sterling work putting extra RAM in PCs, showing people how to reboot, and crawling around under desks plugging in errant network cables. One nice thing about working in IT is it keeps you from getting too grand an idea of yourself.

I'm still not hearing about any programming.

Well, over time, and with a certain amount of sheer persistence, I managed to parlay my sysadmin duties into some actual software development. You know the kind of thing: it started as a five-line shell script for running backups, and in a couple of years it was a 5,000-line spaghetti Perl abomination that ran the entire business, and no one but me understood it. (Not *even* I understood it, of course, but that didn't stop me tinkering.)

It was the perfect recipe for job security: if you write bad software that people depend on, you can make yourself indispensable.

I might edit this part out too.

I'm here to drop truth bombs. But even though I was developing a lot of software, I wasn't officially a software developer. That came later—

Let's pause at that point, to generate some artificial suspense and excitement.

Really? Well, okay.

Don't miss the next part, where we'll learn more embarrassing and discreditable secrets about John's past.

Wait, what?

2. Not a real developer



They said I threatened someone with a knife. That's an absolute lie. It was a screwdriver.

In the first part of my chat with the reliably on-point [Zack Proser](#) about my horrible career, we heard how a degree in computer science was the easy option for a coding-obsessed geek, but it didn't translate into a lucrative software engineering job (well, that sucks).

Instead, I toiled in the word mines for a few years as a tech writer, cranking out user manuals warning people not to dunk their personal digital assistants in water. Over to Zack:

John, tech writing is important work, as we all know. But it must have seemed a little unsatisfying to someone who always really wanted to be a programmer. And you'd already been writing code for a long time at this point.

I had: I'd been a hobby programmer all my life, as well as mixing with lots of professionals over the years: that is, people who actually had "software engineer" on their business cards. In many cases, I had as much as two decades more programming ex-

perience than they did, which made it all the more frustrating that I couldn't get anyone to pay me to do it.

There *is* a big gap between recreational programmers and professionals, to be sure. It's not so much about the code you write as the *way* you write it: things like tests, version control, program design, abstractions, APIs, and so on.

I didn't know any of that stuff back then, but of course I didn't *know* what I didn't know. You never do. All I knew was, I felt like a software engineer trapped in a sysadmin's body, yearning to be free.

I can relate to that feeling. When I was working in technical marketing, somebody once described me as “not a real developer”, and that burns me to this day.

What FUCKER said that?

Gosh, that's about the meanest thing I've ever heard, and I'm on *Twitter*. There's no excuse for that kind of gatekeeping.

You should have given him some snappy comeback, like “Oh yeah? Well, [the jerk store called](#). They're running out of you!”

I wish I'd thought of that.

You will, Zack, you will.

Well, I still managed to learn a lot about programming, partly from going to the pub with my “real” software engineer co-workers. (Sidebar, they drank a *lot*. I didn't know why, then, but I think I do now.)

Also, like most sysadmins and infrastructure people, whatever my business cards said, I was *writing* a ton of software, for users who depended on it.

So, for all those out there suffering from imposter syndrome because your company doesn't call you a “real developer”, that's bullshit. If you write software, you're a software developer.

Thank you! That's what I'm saying.

For example, I wrote an installer for the company's main product. That wasn't trivial, because it was a server-based thing, and it involved checking for things like “do we have the libraries we need, and can we install them,” and “which version of the product works with this operating system,” and so on.

Well, that company eventually tanked (not because of my code, as far as I know). But now that I had some proper developer experience on my CV, I managed to get a job as a proper developer (using Python, as it happened). It was great. I felt like Pinocchio becoming a real boy.

It helped that the company were trying to port their product to Unix, and I was the only applicant who knew what an inode was, or how to *fck* a filesystem. Good times!

There's a bit of a Unix theme emerging, no?

Right, and that knowledge was acquired in an extra-curricular way. I wouldn't have learned it if I'd stuck to the textbook. When the other kids were learning Pascal, I was the smart-alec who wanted to do the assignment in C, and found a battered old [K&R](#) book at the back of some cupboard, like the Half-Blood Prince's copy of "Advanced Potion Making" by Libatius Borage.

You can't learn C without learning *something* about Unix, and vice versa—the two things are just so entwined. That's true of Go as well, by the way. It has a lot of Unix DNA (and is clearly the spiritual successor of C, to the considerable annoyance of lots of people who don't like C).

What is the Unix gene, then, would you say?

Richard Gabriel coined the phrase "[worse is better](#)", which sums it up well. Simplicity is paramount. Simple systems are easy to write, easy to port, easy to use, and easy to extend. Simplicity beats even strict correctness, in this view: it's better to be simple (and handle the easy 90% of cases in a nice way) than to be totally correct (and handle the awkward edge cases, at the expense of making the code much more complex).

That sounds familiar to Go programmers, right? Go is almost brutally pragmatic. It's not a beautiful language at all, at least on a surface level. Programming language theorists hate it! It may be fine in practice, they say, but it'll never work in theory.

Go is designed to be easy to parse, quick to compile, and simple to implement, just like Unix itself. It's not aimed at programmers who want to express themselves in rich and byzantine hierarchies of type systems, or who need the compiler to catch all possible bugs. It's just a plain, non-fancy language for getting useful programs written and running as quickly as possible. I appreciate that about it.

So you were a Unix sysadmin, slash Perl & Python developer, slash network cable plugger-inner. Where did that take you next?

I found myself in a tricky spot. I enjoyed the actual coding, but it wasn't a very good company, and I didn't gel with my co-workers. I thought their way of doing software development was tragically crap, and conversely, they didn't value any of the things I thought *I* was bringing to the party.

Like what?

Tests, automation, deployment, source code management. Software quality in general. All the stuff I'd learned from decades of hanging around with "real" professional developers.

It didn't go well. They didn't see what would be in it for them. When I explained it would result in a better, more reliable product for their customers, the response was "Why would we care about that? We already have their money."

So it wasn't really in the stars, type of thing. Eventually the company and I came to what's usually described as a "mutual decision to part ways", or less euphemistically, I was canned. [*They* said I threatened someone with a knife. That's an absolute lie. It was a screwdriver.]

I know this next part. You became an internationally-recognised consultant, expert on infrastructure automation, and author of such books as [The Puppet Cookbook](#). Right?

No, I was unemployed for a year, got dropped from the books of every recruiter in England, and finally ended up taking a desperation job working nights in a data centre in East London, plugging in network cables.

Okay, it wasn't *always* nights (it just felt that way, as you'll know if you've worked nights yourself), and it wasn't a completely unskilled job either. There was a little light sysadmin involved, but it was a massive, global company. I was the smallest imaginable cog in a very big and impersonal machine.

It was very like [being in the military](#), I suppose: you're not there because you're smart or have good ideas, or even a [particular set of skills](#). You're there to salute officers, follow their orders without question, and most importantly of all, to keep your boots shiny at all times.

In that environment, having good ideas is actually a bad idea. Suggesting things could be changed implies there's something wrong with the status quo; a Catch-22 I encountered again later when I became a consultant. You know, there's the right way, the wrong way, and the Army way. In that firm it was very much the Army way, or, more accurately, the Navy.

So it was an often frustrating experience. And I *still* wasn't a real developer.

Huh. Didn't see you as the saluting type, to be perfectly honest.

That's more or less what I thought, too. That kind of highly structured environment suits some people, I'm sure, but I'm not one of them. Needless to say, then, I wasn't a huge success in this job *either*, though the trench-warfare setting did bring us all very close as co-workers. When you've fought alongside someone under intense enemy fire, doing a tricky disk replacement at 5am on a control server for a [nuclear power station](#), you form a bond with that person.

I did learn a lot about big infrastructure, and automation too. Obviously an operation of that size has to be almost entirely automated. When you just have a few servers or clusters, you can manage them more or less by hand, with a few basic scripts. When you have a few *thousand* servers, it's essential to automate monitoring, backups, installs, upgrades, and everything else. And economies of scale make that kind of investment worthwhile.

So that's when you became a successful consultant and internet personality?

No, it's when I nearly died of malnutrition from living on vending machine food and unbranded instant coffee. Plus the sleep deprivation.

One thing about nights, though, is that not a lot happens, so there's plenty of time to fill. We used to watch movies. (One guy used to watch pornos, so I was always careful to give the chair a bit of a wipe down when I took over his station.)

Or we would play games, read websites, chat with people in other timezones, grab a few minutes of poor-quality sleep in a chair (this is physiologically impossible, as anyone who has tried it will know). I did a lot of hobby programming.

And you did a little moonlighting, perhaps?

Someone I'd worked with in a previous company got in touch to ask for some sysadmin help, on a contract basis, and I thought "Why not?" After all, I had plenty of free time at nights: essentially, we were just there to weigh down the desks and stop the AC from blowing them away.

So I would do the contract work remotely at nights, while nominally on shift for my *real* job (sorry, supervisor), and visit my client's site on weekends or off days to do things like plugging in network cables.

Is that when you decided to take the leap and strike out on your own, then?

No, because I didn't get any other contract work after that. But when my *actual* employer and I came to the inevitable parting of the ways (I had my own version of Calvin's "[noodle incident](#)"), I got a job in another company that was a lot nicer to work for (and no night shifts).

For various reasons, most of them to do with evading taxes, everybody at this firm was technically a contractor, not on salary. And, again for reasons specific to UK tax regulations, we were encouraged to take on other contracts at the same time, so that we wouldn't look *quite* so much like de facto employees.

I already had my limited company in place—not that you need one to do freelance work, but it's nice to have—so I reached out to a few contacts, and little bits of work started to come in here and there. A friend and co-worker (who's now an extremely Distinguished Engineer at a well-known company, but I knew him *before* he was quite so distinguished) did me what turned out to be a very consequential favour indeed, by introducing me to [Puppet](#).

What's that?

In the bad old days, managing configuration on servers and networks was anarchy, but anarchy made of a lot of largely non-portable and incorrect shell scripts. Every company just cobbled together some bespoke tools (one was literally called [Cobbler](#)) and did their best.

The problem is, you can *build* a machine with a shell script, but then what? If you make manual changes to the server, and then forget about them, there's no automated way to revert them.

And you can't easily roll out a single change to a big farm of heterogeneous machines, all with different operating systems and software versions, in a safe and repeatable way. That's the problem Puppet solved so well, by turning configuration into software: *infrastructure as code*, to coin a phrase.

I thought that was called "Ansible".

Never heard of it. Anyway, managing infrastructure has always been about writing and maintaining software of various kinds, as we discussed, but now there was an actual programming language specifically tailored to the job. What you'd need to get the most out of it would be someone who knew about Unix, servers, and networks, but also how to write good software.

All of a sudden, then, there was a me-shaped niche in the industry.

So *that's* when you went fully independent?

No, that happened later, when I got fired... again.

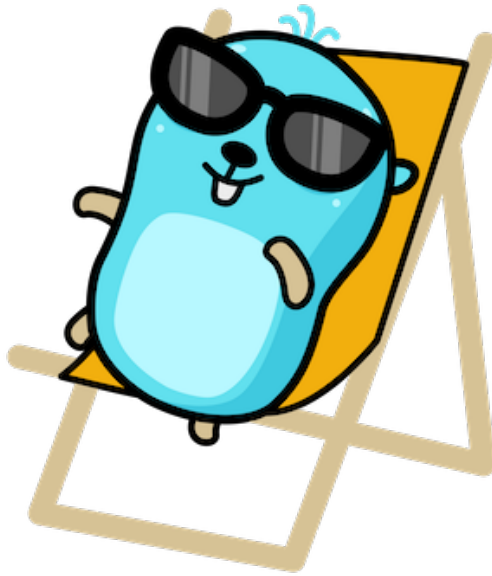
I can see why you didn't mention any of this in your careers book. Shall we pause there, and pick up the story of what you did next, next time?

Sure. We'll talk about how I became self-unemployed, and what I learned about running my own business, by my standard method of making every possible horrible mistake and suffering the consequences.

Sounds good, my man. Catch you on the flippity flop!

I have no idea what you just said.

3. Master of my domain



Business Jack does not play gentle

Devex supercharger [Zack Proser](#) and I have been talking about my history of bouncing around the tech industry like a pinball, the times I went on tilt, and how I (mostly) avoided going down the drain. (Wait, do people still play pinball, or is that a bad reference? [How do you do, fellow kids?](#))

This time, we're talking about how I made the leap from being a (clearly not very good) employee to running my own independent business.

So, when we spoke previously, you were becoming a master of Puppet, writing code to manage servers and networks. What happened next? Promotion?

Nope. Redundancy. A manager at the company asked the question “[Why do we need all these sysadmins, when the servers never go down?](#)” Now, there's a very good answer to that question, but somehow they didn't seem that interested in learning what it was. (They found out eventually, though.)

Although technically I was self-employed, this company had been my only significant client, and so without them, I became self-*un*employed. Oddly enough, it was the biggest favour they ever did me.

Because you like living on ramen?

Sure. With lots of black pepper, soy sauce, and cheese, you've basically covered the four food groups. Yes, those were thin times for a few months, but little by little, work started to trickle in. It's not so much that people wanted to hire me to do Puppet work, as that they had config management problems that needed solving, and I could use Puppet to solve them.

Indeed, for a while I could bring Puppet into companies and it made me look like a genius. "Here's the code you need to set up every server in your network. No more manual builds. You're welcome." I didn't really know anything, of course, but I *seemed* smart because I knew one thing they didn't: infrastructure should be code.

As a consultant, I would go into a company, figure out what they needed, and write software to make it happen. They were delighted, because instead of having to hire a traditional server wrangler to keep things running, they had a more or less self-managing system that their own developers could tweak when they needed to.

Within a few months I actually had a decent base of established clients, and enough new business coming in to keep the account book healthy. So, although the prospect of going it alone was scary at first, and success wasn't guaranteed, I pretty soon reached the point where I started to think "You know, this might just work out."

So what was it like running your own business? Was it what you imagined?

Yes and no. I mean, working for someone else never really suited me, and that's something all my erstwhile employers would probably agree on. I'm just one of those people who doesn't thrive on being told what to do by others. Unfortunately, that's something we all have to deal with in life to *some* extent, whether we like it or not.

It's always been a challenge for me. I can remember being thrown out of the Cub Scouts at a very early age—though I can't remember exactly why.

Failure to salute an officer?

Yeah, probably. So maybe some people are just made that way: they don't respond well to authority. Those are probably the people who are going to want to be in charge of whatever organisation they're in, and who aren't really going to be happy until that's the case. That's annoying if you're supposed to be the boss of that person!

You're unmanageable.

That's so, and when you run your own business, you soon realise how much managing *yourself* is going to be an issue. You know, you can get it wrong both ways. There's no one telling you when to work or what to do, so you can drift or get distracted, and waste your time. Or you can drive yourself too hard, working every minute of every hour, always stressing out that you could be doing more, so you *should* be doing more.

I'm shocked. Shocked to hear that independent working isn't just a matter of sitting in a beach chair, sipping Mai Tais.

That's *part* of it, sure. I mean, I'm a little drunk right now. But there's also a lot more on the negative side than I imagined there would be.

Example?

Life as a salaried worker is very simple, in some ways: you just show up at nine, do your assigned tasks, and go home at five. If you work from home, you don't even have to get dressed (maybe just the top half). You don't, for example, have to worry about finding customers, marketing the company's products, sweet-talking investors, dealing with tax and payroll, or fixing the phones. You have the exquisite moral luxury of never compromising any of your principles.

When you're a developer, especially, what you don't realise is that a large part of the company's workforce is devoted entirely to making and keeping you happy. It might not always feel like that, but you're in a very feather-bedded position. If something goes wrong with facilities or logistics, you just open a ticket and someone takes care of it. You have layers of management to protect you from the crap that comes down from on high. If the company has a rough quarter and doesn't make any money, you still get paid. (For a while.)

You're thoroughly insulated from the occasionally dirty business of... well, business. A company can't exist without making money, and I've known some software engineers who turn up their delicately-shaped noses at this idea. "Filthy lucre? That's all right for those MBA types upstairs, but I'm here to make the world a better place."

Sure, the world legitimately couldn't get by without your cat-picture sharing app, or whatever, but what you don't see—or don't want to see—is that someone has to pay your salary. The money for your nice laptop, your Herman Miller chair, and your cafeteria snacks has to be grubbed up by someone actually sending invoices and making sure they're paid.

Software engineers feather-bedded? I literally can't believe what I'm hearing.

Sometimes business can be a bit rough and tough. It's easy to ignore that when someone else is taking care of it all. When *you're* in charge of the business, it suddenly looks a bit different.

Just because you're nice doesn't mean everyone else is nice. People will try to rip you off. You've got to be able to deal with that. Or even nice people will sometimes give you the runaround.

What if your client doesn't pay their bill, and every time you chase them about it, they just fob you off with some excuse? Can you get tough with them in the right way? Not shouting and screaming, but being assertive enough to get the message across, without putting your future relationship in jeopardy.

JACK: It needs to be communicated to Lemon that if she wants to play business, I'm more than happy to play with her. But as she saw earlier today, Business Jack

does not play gentle.

JENNA: *Are you as turned on as I am right now?*

—“30 Rock”

I’m a Kenneth myself, but I actually saw you as more of a Lemon.

In private life, I definitely am, but as the boss I have to channel Business Jack a bit, without *becoming* him. The thing is, there’s no online course for this. (Well, there are probably plenty, but they won’t tell you the stuff you really need to know.)

Like?

Early in my consulting career, I had a contact from a very promising-sounding client. They were building out a big music-based social app and wanted me to design, architect, and scale the infrastructure. They asked to meet and discuss, to see if I was the right guy to help them.

I had plenty of experience in this area, so I figured I’d have no problem landing the gig. We met in a swanky hotel bar in London, which I now realise is a red flag. If the company doesn’t want you to come to their offices, it’s because they *have* no offices. But whatever. I knew they were a startup that hadn’t started up yet: that’s one reason they needed my help.

We had a terrific meeting that lasted about two hours. Their business deck was good, and their expectations about scale and cost were realistic. I said, “Okay, here’s what I’d recommend. Your infrastructure block diagram would look something like this...” (*sketches*) “...and you should go with cloud provider X, because...” (*reasons*) In essence, I designed the whole thing for them right there at the table, with all the tech stack info and performance numbers they’d need.

I have a bad feeling about this.

I chuckled to myself all the way home from that meeting, thinking “I crushed it with those guys! That’s the best two hours of consultancy I’ve ever done. They can’t fail to sign me up now.”

And you never heard from them again?

And I never heard from them again. They *had* everything they needed, and it didn’t cost them a penny. My “hire Bitfield Consulting” pitch was *too* good! In my eagerness to show them how much I could do for them, I *did* it for them. That’s the kind of mistake everybody makes as a consultant, especially if you’re a decent, honourable sort of person by nature. *You* wouldn’t rip people off like that; the idea simply wouldn’t occur to you. So you’re not looking out for it when it happens.

Integrity is great, and ultimately it’ll be the key to your success in business: if people can’t trust you, they won’t hire you. But you also need to acquire a bit of low-down cunning. You have to know how the dirty tricks are done, so that you can avoid people doing them to *you*.

This is gold, John. Gold! Anything else?

Here's another pearl I gleaned the hard way: don't take fixed-price contracts.

The first consulting work I ever did, the guy said "We can pay you X grand for the whole job". Nice and simple. Everybody's a winner! Or at least I thought so, until after much labour, I presented them with the finished system.

And they paid you on the spot, right?

...

They paid you on the spot, right?

You know, that's the funny thing. They said "This is great, but actually, we need one or two little changes." I made the little changes, but they still weren't happy. "Some of the requirements changed since we gave you the spec, so you'll need to replace X, Y, and Z." And so it went on.

It took me quite a while to realise that, ever since I delivered the system they initially asked for, I'd been working for free. After all, each change request was small in itself: not worth jeopardising the deal over.

They always made me feel I'd screwed up a little bit, but it was okay: they were prepared to forgive me, so long as I made it right... for no extra money, of course. In business terms, it was an abusive relationship.

"Baby, why you always gotta make me hit you?"

I mean, it makes perfect sense from their point of view. It's just easier for a business to sign off on a single purchase order for a fixed amount, rather than commit to an unknown and variable ongoing cost, with no end date in sight. And, having agreed the price in advance, they then had every incentive to squeeze everything extra they could out of me for free.

They had total power over the contract, because until they said they were happy with the deliverables, I didn't get paid a cent. I was at liberty to walk off the job at any time, of course, so long as I didn't mind writing off all the money I'd worked so hard to earn.

And, I need hardly add, I *minded*.

You got played, my brother.

So hard. And I don't even bear the guy any malice. He was just a little sharper at business than I was, that's all. Well, that's how you *get* good at this stuff: by screwing up, and learning from it. I learned a lot from that engagement, and all the others.

My mistakes were many and expensive, but in terms of value for money, it's actually one of the best training courses I've ever taken.

Let's press pause at that point, because I need to go re-think some of my business plans.

No worries. We'll pick this up in the next chat, and talk a bit more about money: what to charge, how to strike the right deal, and how to negotiate (or not). Until then, good waves.

4. If you need the money, don't take the job



Your lovin' gives me a thrill, but your lovin' don't pay my bills

It's late, and the campfire is burning low. My fellow content creator [Zack Proser](#) and I are sitting on a log, chowing down on s'mores and enjoying a few spine-chilling tales of terror about my horrible career.

Well, not that horrible: I now make a living from at least two of my favourite activities, [writing](#) and [mentoring](#). No luck so far getting people to pay me to tinker with their vintage Land Rovers, or to sit in a comfy chair eating chocolates and watching classic *Doctor Who*, but hope springs eternal.

Last time, we talked about how I made the leap from salaried server-monkey to starting my own company. Independence is great, but it turns out that when you quit your job, you also stop getting the regular paychecks (why does no one ever tell you these things?)

Money, like oxygen, is one of those things that you don't really appreciate until you suddenly don't have enough of it. Some of the biggest challenges for the newly independent developer or creator, indeed, revolve around money: how to get people to give it to you, how much to ask for, and how to be *worth* what you're asking. Let's get into that. Over to Zack.

In our last chat, John, you froze my blood with a horror story about what happened when you unwisely took on a fixed-price contract. From what you said, it doesn't sound as though a deal like that is good for either you or the client?

It's not. A fixed-price deal sets up the wrong incentive structure. It's in the client's interest to agree the lowest price possible, and then to squeeze the consultant for every drop of juice they can get. So that sucks.

On the other hand, it's in the consultant's interest to do the quickest and cheapest work necessary to get the client to sign off on the job. After that, they have zero incentive to fix any problems that arise. It's just bad news all round.

So what would you recommend instead?

When someone asks for a fixed price, I'll explain to them why that's not in either of our interests, and suggest that instead they pay me a fair hourly rate for what I do. And, so they know what they're in for, I'll give them a careful and realistic estimate of how many hours it'll take to deliver what they've asked for. That gives them the cost control they need, but doesn't *commit* them to the whole sum right away.

Once we've agreed the terms, I can get started, and I have a strong incentive to deliver value for them right away. If they like the work I'm doing, and they want more, I'll happily provide it: it pays me to do so! On the other hand, if after the first day or two they think "This guy's no good," they can just pay me off and wave goodbye: they're not locked in for the long term to a vendor choice they're already regretting.

At the end of the job, if they want further changes or enhancements, they can have them, but not for free. I'll estimate them, and then they can decide whether or not it's worth the extra cost. That prevents a nasty case of "just one more thing" syndrome.

One reason some companies don't like to pay for consulting work by the hour is that they think it'll incentivise the consultant to goof off. You know, take it slowly, invent problems and delays, anything to rack up a few extra hours. Sure, unscrupulous consultants will do that, but then it's on the client to pick that up and get rid of them. It's not a successful long-term strategy.

Why not?

I've heard it wisely said that there are two kinds of consultants: poachers and farmers. The poacher wants to make a quick kill and get clean away. The farmer, on the other hand, is in the game for the long haul. She shears the sheep closely, but gently, and nurtures it. She feeds it, cares for it, keeps it warm in the winter, and tends it when it's sick. That way, it'll be around for her to shear again next year.

Flattering analogy.

Okay, but I've worked on farms: you'll never find a deeper love than a farmer has for her sheep. They're literally her livelihood: if they do well, she does well. Sure, she might see a quick profit if she slaughtered them all this year, but then there's no *next* year. Instead, she looks after the sheep, and they'll look after her.

Similarly, as a consultant you *could* strike fast, extract as much money from the client for as little work as possible, and then ride like hell for the hills. There'll always be more unsuspecting clients for you to predate upon, won't there?

It seems tempting, but actually, it doesn't work. Hunting for new clients all the time is difficult and expensive, and often it doesn't work out: you put time and effort into meetings, proposals, and estimates, and half the time it just doesn't go anywhere. Much better not to lose the old client in the first place.

Also, sooner or later, the word about you will get around. One way or the other. So you should be the consultant that *you'd* want to hire.

Repeat business is the best business, because marketing is hard. After a while, you won't *need* to market your services. People will start coming to *you*. At that point you know you're doing something right.

I see the value of charging by the hour, but one thing I struggle with personally is knowing *what* to charge. Do you have any advice?

Yes, that's difficult. The best advice *I* ever got on this was "Think of a number, then double it." If you don't feel a little embarrassed when you tell them the rate, that probably means you're coming in too low. If you always get a yes, then you're *definitely* coming in too low.

The tendency is always to under-price yourself. Partly because you're modest and self-effacing (that's one of the reasons we all love you), but also because you want the business. That's another big mistake I made starting out. I thought "Well, I don't really *know* very much. I'll charge a low rate to reflect that."

Here's a weird fact: businesses actually *like* paying a lot for consultants. It's called the [Chivas Regal effect](#). You know, if people see a cheap bottle of whisky, they'll assume (rightly) that it's horrible. But if it costs sixty bucks a bottle, they'll assume it must be a quality product. Otherwise, how on earth could the manufacturer justify charging that much for it?

And the whisky actually tastes better *because* it costs a lot.

It sure does. Cheers!

Imagine a manager has some problem they can't fix. What's their response? Hire a consultant. A cheap consultant? Absolutely not. Only bad managers have cheaply-solved problems.

The more they spend, the greater the kudos they'll get from upstairs, and the bigger their budget will be next year. After all, the division with the most expensive problems must be the most important one in the company.

And everyone can relax and feel good knowing that the problem's being taken care of by the highest-priced consultancy firm available.

Wait, what?

I'm serious. When you're on trial for murder, do you want to hire the cheapest lawyer in the phone book? I don't think so. You go to the *super* expensive lawyer, the one that defends Tom Cruise against whatever the latest weird and concerning allegations might be.

Okay, someone else, not Tom Cruise: I don't want to hear from his lawyers. But that's my point.

So it's about being reassuringly expensive.

It really is. You have to *actually* be good at what you're doing too, of course, or the whole scheme falls apart. What the client's money really buys, though, is that delightful feeling of making the thing *somebody else's problem*. You know, we've turned it all over to a top-tier expert, and now we just don't have to think about it anymore. The more that person charges, the more reassured the client feels.

It's better for you, too, because when they're paying you a lot of money, they put a lot of value on your advice. By contrast, the less they're paying, the less they're inclined to listen to you. You need them to listen to you, and that's why you need to charge top dollar.

In "[Secrets of Consulting](#)", which is the first book everybody should read on this subject, the incredibly wise Jerry Weinberg says:

Make sure they pay you enough so they'll do what you say.

That's genius, but... how do you work out how much that should be?

Here's a good way to get to the right number: every time a client accepts your price, make it a little bit higher the next time around. When people start complaining, or turning you down, you know you're getting into the ballpark. When your acceptance rate falls to about fifty percent, then you're probably there.

And don't forget, the more clients you work with, the more *you* learn. The more you've learned, the better a consultant you are, and the more you can charge. So your price should steadily increase over time (but make sure it's always in line with your value).

What if they want to negotiate?

You can negotiate on everything else, but don't negotiate on price. Would you go to the cheapest dentist? Would you haggle with your brain surgeon from the operating table? I hope not. Some things just aren't worth cheaping out on. Clients will still try, but you should politely decline.

Clients who want a discount always turn out to be the worst clients, if you cave in to them. By offering you less, they're really saying "We don't think you're worth what you're asking." You don't want to work for someone who sees you that way.

Plus, people who ask for discounts are habitual cheapskates. They think they're smarter than everybody else: only suckers pay sticker price. They think you're trying to take *them* for a sucker.

If you accept their lowball offer, now they know they've got you on the hook. They'll try to screw you in every other way, too: asking for extra things here and there, demanding shorter deadlines, out-of-hours support, every little deal-sweetener they can think of.

Unless you really can't survive without that client, don't take them on. Jerry Weinberg also says "If you need the money, don't take the job."

That's great advice. I'm putting "Secrets of Consulting" on my reading list! What else?

One common problem is that the client doesn't know anything about the subject area: if they did, they wouldn't need you. So they also don't know what constitutes a fair price for the work. When they hear your rate, it may sound too high, and you won't get the deal.

But that's okay. You just dodged a bullet, because you don't actually want a client who doesn't understand the value of what you do. That'll never be a profitable engagement, in any sense of the word.

Once someone got in touch with me and, when I quoted my rate, they said "Gosh, that's so much more than we were expecting. We'd like to hire you, but there's a guy in Eastern Europe who says he'll do it for like 20% of that." I said fine. Absolutely, go with the cheap guy. If he can do the same quality work as me for a fifth of what I charge, I told them, then it's positively their fiduciary *duty* to hire him, which they did.

A few months later they came back to me and said, "Now we understand why he was so cheap. Are *you* still available, by any chance?" And of course I was... at the rate I originally quoted.

Business Jack does not play gentle.

That ended up being a long and very rewarding engagement, and they were super happy with my work. Partly because it *was* good, if I say so myself, but also because they'd seen something of the cheaper alternative. They'd learned the value of what I do, and I learned that it's okay not to let yourself be beat down on price.

Things can still go squirrely even after you've landed the gig, too. You know, you give the client the benefit of your expertise, and they say something like "Thanks very much, but that won't work for us. We're going to do XYZ instead, because money / politics / reasons."

It's easy to get upset in that situation, and start arguing with them or berating them. What kind of idiots pay for an expensive consultant and then ignore what they say? The commonest kind, it turns out.

The fact is that they may just not be ready to *hear* what you're saying. Sometimes they need a little time to think about it, and sometimes they need to learn what happens if

they do it the other way. In fact, I've gone so far as to formulate what I call Bitfield's First Law of Consulting:

Sometimes the client needs to feel the pain of not taking your advice, before they're ready to take your advice.

It's amazing how effective that can be. When they *do* come back to you, it's often with a greatly increased respect for what you say next. "He said we'd crash and burn if we did X, and he was right! This guy really knows what he's talking about. Let's listen to him in a rapt, respectful silence."

I'm doing just that. Shall we take another natural break, and return for a final bite of your wisdom s'more later on?

Sure thing. We'll talk about how my consulting career ended up taking me somewhere quite unexpected, but that turned out to be exactly where I needed to be. See you next time.

5. Will write for food



The secret of being happy ever after is not to be after too much

This is the end, beautiful friend.

In this series of interviews, I've been sharing some war stories about my horrible career with [Zack Proser](#): how I got into this business, how I made it *my* business, and also how I made my seven hundred worst mistakes, so that you won't have to.

And so here we are at the end of my career. Not that it's *over*, quite yet: it's just where I want it to be. I'm running my own company that supports me and my family. No one can tell me what to do or when to do it. I have the freedom to work on, learn about, or write about whatever I want, providing I can occasionally turn that into money.

I no longer feel that life is passing me by, while I dance to somebody else's tune. Instead, I'm living it on my own terms. So what does that look like? How do I pay the bills as an independent writer, teacher, and content creator?

Hey, I'm asking the questions here.

Sorry, Zack. I didn't mean to put words into your mouth. What did you want to ask?

John, we've talked about how you became master of your domain, and some of the lessons you learned from being a consultant. How did you get from there to being a full-time writer / mentor?

Well, I enjoyed my ten-year spell of consulting very much. I mean, I've always liked telling people what to do, but up to that point I hadn't really considered it as a career option.

And I wasn't really a very good consultant in an economic sense, because instead of fixing peoples problems, I taught them to fix their own problems.

So they didn't need to call you again after that?

Exactly. On the face of it, making yourself redundant seems like a poor business strategy. On the other hand, though, some of my clients seemed to value my work so much, they kept on creating new problems they needed my help to solve. Plus, they'd tell everyone they met what a terrific consultant I was. (I think that says less about me than it does about the *previous* consultants they'd dealt with, but never mind. I'll take it.)

And I enjoy working with smart and capable people over a long period, passing on some of the technical tips and tricks I've picked up over the years, and seeing them develop their own skills and experience. It's really fun.

You're starting to sound like some kind of... mentor.

I know, right? They say an expert is just someone who's one page ahead of you in the manual, and that's how it was. I'd learn enough to help the client fix their problem, and I'd show them everything I'd learned. Once they knew as much as I did, I could move on to the next person who needed my help.

Explaining complicated things simply is a skill, too.

That's very true. Apparently I have it, to some degree. You never know what you're good at, of course, until someone tells you that not everybody else can do that particular thing. Once you find that out, the idea occurs to you: maybe I could do this for a living!

And you've written a few books over the years in which you explained complicated things, starting with *Puppet*. When did you decide to write your first book?

It was the nice people at Packt Publishing who suggested that, and very flattered I was, too. I now realise they probably talked to a dozen or more people before me, but *they* were all too busy, or too rich, to take on the project. *I wasn't either of those things.*

In fact, writing the book was a lot easier than I thought it would be, because I'd already had so much practice explaining this stuff as a consultant. After a bit of trial and error, you work out how to introduce the different topics, how to lead from one thing on to the next, and so on. All I had to do was explain the things that I wished *I'd* known when I first got started.

Often, getting an idea across to someone is a matter of finding just the right form of words that lights up the person's brain. Again, you try different ways of putting some-

thing, and after a while if you're lucky you'll find one that seems to work with many different people.

All I did was take the best of the explanations that I'd gradually honed on the road, and stitch them together into a book, like a comedian putting together a tight five.

And you killed, John. You killed!

Well, I wouldn't say that, but I'd be willing to at least listen politely while *you* said it.

At any rate, the first book was successful enough to lead on to other Puppet books, and teaming up with my friend Justin Domingus to write [Cloud Native DevOps with Kubernetes](#) for O'Reilly. Again, we just wrote the book that we wished we'd had, when we were wrestling with Kubernetes and drowning in documentation.

I really like that book. It's so friendly and matter-of-fact. Technical books can be kind of stuffy sometimes.

Thanks, I appreciate that. The funny thing is that several people who know me said that when they were reading the book, they could hear the words in my voice!

And if they said your name three times, you'd appear in the mirror?

Only if they're very unlucky. Anyway, I enjoyed the writing process a lot, and, if I'm honest, I liked seeing my name in print. I think everyone does. At that point, I had the book-writing bug, and since my main focus by this time was on Go, I decided I'd like to write about that.

What made you switch to self-publishing? Is that something you'd recommend for people interested in authoring books, or should they go the traditional publisher route?

Well, I did have some offers from various publishers inviting me to write Go books, but I turned those down, with some trepidation. It's always tough saying no to money, but sometimes it's the right thing to do, as we discussed last time.

It's not that I hated working with publishers: it was a good experience overall, but what I always wanted was a bit more control. You know, the publisher basically decides the subject matter of the book, probably the title, certainly the length, and most likely the cover design. They also set the price.

Really, the writer is quite a small part of the whole production, in terms of creative control. You're essentially a hired hand on someone else's project, and there's nothing wrong with that in itself. But to my mind, in that situation, it's not *your* book. It's theirs.

And you've mentioned once or twice how you chafe at being told what to do by someone else. A few of your annual reports said things like "Not a team player".

I admit it freely. It's not that I always think I know best: that would just be narcissistic personality disorder. It's more that I like to stand or fall by my own efforts. If I do good work, great; if it's not so good, I'll take that on the chin and figure out how to make it better. The responsibility is mine, for better or for worse.

I can relate to that. If you're going to become a master of some craft, whether it's writing or coding, I think at some point you've got to strike out on your own and practice it. You need to have skin in the game.

Very true. But not straight away! I mean, I did my apprenticeship, in a sense: I worked for other people for about fifteen years before I felt confident enough to go it alone. Even then, I was a bit doubtful, but you just can't wait until you feel that you've mastered the craft before you get started—you'll never feel like that!

There's always more to learn, but that doesn't mean what you've already learned isn't valuable: it all is.

At some point, as you say, you've just got to plunge in and struggle like hell until you make it. But you will make it.

That's reassuring. I mean, I still feel like I don't know enough, technically speaking. Why should anyone pay to hear what I have to say?

Yes, we all feel that way, I'm sure. Again, though, you never really appreciate the value of the knowledge and skills you have until you come to pass them on to someone else. And you catch yourself thinking, "Doesn't everybody know this stuff?"

No, they don't. No one's born knowing anything, and learning is hard. Getting some help and guidance from someone who's a little way ahead in the manual is really valuable.

And that's a career, right there, if you want it.

It's certainly worked for you. So what does the technical side of your book and course creation look like? I know the words flutter down to you like snowflakes from heaven, but how do you actually turn them into a saleable product, and sell it?

Well, if you're okay with just digital content, that's fairly easy: we all have the tools on our desktops to produce nice-looking PDFs and videos, and so on. I use [Pandoc](#), which is an absolutely incredible piece of software for converting text between a zillion different formats.

In my case, I write Markdown (in [VS Code](#), another of my favourite tools), plus a little embedded [LaTeX](#) for equations, precise layout, and so on. [Pixelmator](#) takes care of my rudimentary graphics, cartoons, and cover designs. Then I use Pandoc to produce a PDF and upload it to the Squarespace store.

One of the many nice things about being in control of the whole process is that I can update the books as often I want. Any time I find a typo or a mistake, I can fix it. Any time I have a better idea about how to say something, I can tweak the book and have the updated version on sale in minutes.

And, of course, Go is being updated all the time, too. So I'm constantly revising the code examples and the text of the books, to track those changes. As soon as a new Go version is released, I go through all the books checking what needs to be updated or added, and I aim to upload the new editions within a few days.

What about people who've already bought previous versions, though? Are they just out of luck?

Not at all (I'm glad you asked that, Zack). All my books come with free updates for life, which no one can quite believe when they first hear it, but it's true.

We've all had the frustrating experience of buying some expensive tech book and finding that it's out of date within a few months. Well, I hate that too, which is why you'll never have to pay again for one of mine. Any time you like, you can just re-download the book from my store, and you'll always get the very latest version, at no extra charge.

It's something that most publishers don't offer, for obvious reasons: they *want* you to keep buying the book every year! Maybe I lose out on a little revenue this way, but I hope that the increase in reader satisfaction makes up for it.

Because all you need to make a living is 1,000 true fans.

That's right. And you'll only get those true fans if you deserve them: if you put absolutely everything you have into making the product the best it can possibly be. And as you get better at what you do, that should feed back into improving the products you already made.

There are basically two ways to make money from selling products. One is the *volume* strategy: the product doesn't have to be that good, and your profit margins can be small, so you can sell it cheap. But you have to sell it in vast numbers for this to work.

I don't have the kind of resources you need to succeed in a volume sales business, and neither do you, or anyone else in our position. That's why I use the *premium* strategy instead.

If you don't sell many units, then you need to make a lot of profit on each of them. That means the price must be high, so if you're to succeed the product must be really top quality. Essentially all independent content creators are in the premium product business, whether they realise it or not.

Like all craftspeople, I suppose.

That's true. I don't think any of us could knowingly do bad work. Otherwise, how would we sleep at night? We do what we do because we want to do it the best way we can. Even if I thought I could make more money knocking out cheap crap, I wouldn't want to.

As we discussed in our chat about consulting, repeat business is cheaper than getting new customers, so quality is the best business plan. And when you're just one person, you're very dependent on your customers to spread the word for you.

You know, if people really like a book, they'll recommend it to all their friends. And when you're an independent author, that word of mouth is all you have. When your stuff isn't on Amazon and you don't have a big company backing you, you can't afford expensive marketing and advertising.

Your marketing team, in effect, is your customers. So you better make them happy!

I see what you mean. So, ebooks are great and all, but what about printed books? They just seem more real, somehow.

I agree. Getting stuff into hard copy is more difficult, though, because it turns out short-run book printing is *really* expensive. Like twenty or thirty bucks a unit, or more, if you want colour and high-quality paper (and you do).

I did try this for a while, and since there's only so much I can reasonably *charge* for a book, the printing costs cut my margin to basically zero, or a little less. In fact, I did make a small loss on every physical book I sold, what with fulfilment and so forth, so it wasn't really a viable long-term strategy. And, of course, I wouldn't be able to offer free updates, as I do with the digital books.

How do publishers do it, then?

Well, I suspect one definition of "publisher" is "someone who has a very favourable deal on printing". You know, O'Reilly or someone like that shifts a lot of books, and they can get volume pricing from a printer, or just buy the printing firm themselves. A print-on-demand machine is expensive, but once you've bought it, you can print as much as you want, and you only pay for paper and ink.

As much as I love physical books myself, and I have a cottage full of them, I can't actually make a living printing and selling them. I *can* do that with digital books, because the overheads are so much smaller.

Such as? Can you share some figures?

Well, it costs about \$20 a month to run my [Squarespace](#) website, which includes the e-commerce store, and another \$25 for my [EmailOctopus](#) mailing list. For each book or course that I sell, Squarespace takes a 3% cut, and the PayPal or Stripe fee for the payment is about 5%.

If you figure that the website and mailing list and other business expenses account for about another 2% of the cover price, then that's a net profit to me of 90%. Compare that with the revenue split you'd get from a traditional publisher, which is probably more like 10%.

But maybe 10% of a bigger number.

Maybe. If the publisher decides to put any effort into marketing your book, which they might not. And if it doesn't do well in the first quarter or so, they might lose interest and withdraw it from the market. Either way, like everything else, it's out of your control.

I'm not denying that it's nice to get little dribs and drabs of publisher's royalties from time to time, and even though it's not much, it's basically free money (unless you count the hundreds or thousands of hours of unpaid labour you put in to writing the book). But you can't *live* on it.

If you want to actually make a living from creating content, you need to own it. Once you sell or license it to someone else, you're back to just being a hired hand, working to boost someone else's profits, and that's not for me.

And do you make a good living? Or are you just getting by?

It varies. Sometimes, for no particular reason that I can work out, I have a good sales month, and a decent bit of money comes in. Other times, again mysteriously, days will go by without me making a sale. You can't look at the analytics too much, or you'd go crazy.

Ultimately, you just have to ask yourself whether you're doing good work. If so, keep on doing it, whatever the analytics say. If not, do better. You're the only one who can judge the quality of what you're doing: that's what it means to pursue a craft.

On average, though, I do all right. I can pay the bills, and there's an awful lot of people who aren't so lucky. I have my freedom, and that counts for a lot. I also have pretty low overheads. They say the secret of being happy ever after is not to be after too much.

Basically, running an independent business is never going to make anyone a ton of money. That's not what it's about. If that's what you want, you should go and get a job.

Just be aware that what you're selling for that fat salary is, essentially, your life. And, sooner or later, you'll realise that you can't buy it back.

Stirring words, my friend. But you're right. What good is money if you haven't got the time to spend it?

Exactly. I have an incredible lifestyle, one that would be the envy of every wage slave if they knew about it. I work when I want, *on* what I want, and I'm not wasting a single one of my precious days on stuff that doesn't matter.

And, if you enjoy what you're doing, is it really "work"? I'm learning new stuff myself and writing software, which is the most fun in the world. I'm writing, which I've always really enjoyed, and I think I'm getting better at it, slowly. And a big chunk of my time is taken up with mentoring, working directly with students, teaching [Go](#) and [Rust](#), and that's probably the most rewarding activity of all.

How much time would you say you put in? Do you work longer or shorter hours now that you're doing it for yourself?

Well, Rich Hickey says that programming is not about typing, it's about thinking. The same applies to writing. You know, if I stand there and rattle at the keyboard, it *looks* like I'm doing something, but that's really only the final stage of the process.

As the avid fans of your [blog](#) and [online courses](#) will surely agree, you're a pretty talented writer yourself, so you'll know that it's not really nine-to-five work. Sometimes you don't get any ideas, and you just stare glumly at a blank editor window, wondering how soon your health insurance is going to expire. Days can go by and you don't get anything written at all.

Other times you find yourself in the zone, and the stuff just flows. Either way, I think you have to accept what comes. With any kind of creative work, you don't really know *how* you do it. It just does itself... or doesn't.

You can't just press a button that says "Make writing happen now!"

That's right. It's more like surfing, which is what I usually do on the days when writing doesn't decide to happen. You don't always get waves: all you can do is be in the right place when they do come. Even then, you have to be able to catch them. A lot of the time you won't, and that's okay.

As a content creator, your prime and only asset is your brain. You have to look after it, and get to know its little foibles. Like an old Land Rover, you have to know when to run it and when to rest it. Sometimes it's reluctant to start, especially on cold mornings, and it needs a little coaxing.

I get my best ideas when I'm nowhere near the keyboard, actually. Instead, I'll be lounging in the garden, watching the birds, or just enjoying the clouds scudding across the sky, and something will pop into my head. All of a sudden, that problem I've been worrying away at has just solved itself. Or I've had an inspiration for my next book.

So if you're trying to write a book, or even a blog post, and you're finding it hard going, take a break. Don't grind yourself into misery and feel like a failure. Instead, do something else, to give yourself the time and space to think.

If you don't open your mind once in a while, after all, how can you expect ideas to get in?

The space between the notes is where the music happens.

Very true. In a salaried job, it's all about looking busy. Your boss will never tell you something like, "Don't just do something, sit there!" But you can't just keep extracting stuff from your brain until it crumbles like a dry husk. You have to put nutrients back into it from time to time, you have to regularly rotate your crops, and you have to occasionally leave it fallow so that the soil can regenerate.

The three things I do, learning, writing, and teaching, are all interdependent. I learn something, and figure it out. I teach it, and by teaching, I learn it at a deeper level, and by trial and error I work out how to explain it to someone else.

Finally, I write about it, and that opens up a whole *new* set of questions, and so the cycle goes round again. But at every stage, I need to give things time to percolate, and that means giving myself the time, space, and silence to think, or to do other things.

Often, the most important thing I can be doing is nothing at all. I've written a [book about that](#) too.

What with the writing, the surfing, the bird watching, and the cottage full of books, I have to admit it does sound like a pretty good lifestyle.

The pay may be modest, but the hours are great. Speaking of which, we should get back to our respective business empires, shouldn't we?

I've enjoyed our chat a lot, Zack. Thanks for taking the time to interview me, and for being such a good listener.

It was my pleasure! And thank you for being so honest, about your screw-ups as well as your successes. I've found it very encouraging.

I'm very glad to hear that, and for anyone who's read this far, I hope it's been helpful. I wish you the best on your own journey to independence, if that's where you want to go. Even if there are a few bumps in the road, you'll make it. Just don't give up.

Well, it's time I was off to the beach. Want to come with?

Count me in, man. Count me in.

About this book

What I do

I'm a teacher and mentor of many years experience, specialising in software engineering, career skills, and life coaching for developers. I've helped literally thousands of people with friendly, supportive, professional mentoring, and I can help you too, if you like. Come and see me at my website:

- [Bitfield Consulting](#)

Feedback

If you enjoyed this book, let me know! Email john@bitfieldconsulting.com with your comments. If you didn't enjoy it, or found a problem, I'd like to hear that too. All your feedback will go to improving the book.

Also, please tell your friends, or post about the book on social media. I'm not a global mega-corporation, and I don't have a publisher or a marketing budget: I write and produce these books myself, at home, in my spare time. I'm not doing this for the money: I'm doing it so that I can help bring the power of Go to as many people as possible.

That's where you can help, too. If you enjoy my writing, and you'd like to support it, tell a friend about this book!

Free updates to future editions

All my books come with free updates to future editions, for life. Make sure you save the email containing your download link that you got when you made your purchase. It's your key to downloading future updates.

When I publish a new edition, you'll get an email to let you know about it (make sure you're [subscribed](#) to my mailing list). When that happens, re-visit the link in your original download email, and you'll be able to download the new version.

NOTE: The Squarespace store makes this update process confusing for some readers, and I heartily sympathise with them. Your original download link is only valid for 24 hours, and if you re-visit it after that time, you'll see what *looks* like an error page:

403

Your download link has expired. Click [here](#) to receive an email with a new link.

But, in tiny print, you'll also see a "click here" link. Click it, and a new email with a new download link will be sent to you.

Join my Code Club

I have a mailing list where you can subscribe to my latest articles, book news, and special offers. I'd be honoured if you'd like to be a member. Needless to say, I'll never share your data with anyone, and you can unsubscribe at any time.

- [Join Code Club](#)

Code For Your Life

[Code For Your Life](#) is a practical, readable, and funny guide to tech careers, distilling the lessons of a lifetime into the ultimate guide for anyone who wants to succeed in this industry.

In these pages you'll learn the key skills for finding the right job, nailing the interview, cracking the technical test and the take-home task, and succeeding in the job despite impostor syndrome.

Where is your career going? How can you plan and negotiate your promotions, pay rises, and contracts? How do you become a master of your technical craft, whether that's programming or something else? Does your future lie in management or on the staff engineer track? When is it time to launch your own business, and how can you succeed as an independent engineer?

The message of this book is empowerment: you can and should take control of your career progress, your skills development, and your working life. The book is crammed with wise, memorable, and often amusing advice, anecdotes, quotes, tips, tricks, and guidance for engineers at every stage of their career.

"I wish I'd had this book twenty years ago," said one rueful reader, but wherever you are on life's journey, the inspiring takeaway of *Code For Your Life* is that it's not too late. You

can recapture the magic and excitement of your first steps in programming, and use it to build a fun, rewarding career.

The ultimate goal, as the book reminds us, is to build a life you won't need a vacation from. And what a delightful prospect that is.

For the Love of Go

[For the Love of Go](#) is a book introducing the Go programming language, suitable for complete beginners, as well as those with experience programming in other languages.

If you've used Go before but feel somehow you skipped something important, this book will build your confidence in the fundamentals. Take your first steps toward mastery with this fun, readable, and easy-to-follow guide.

Throughout the book we'll be working together to develop a fun and useful project in Go: an online bookstore called Happy Fun Books. You'll learn how to use Go to store data about real-world objects such as books, how to write code to manage and modify that data, and how to build useful and effective programs around it.

The [For the Love of Go: Video Course](#) also includes this book.

Further reading

You can find more of my books here:

- [Go books by John Arundel](#)

You can find my blog posts and articles here:

- [Bitfield Consulting](#)

I have a YouTube channel where I post occasional videos on Go, and there are also some curated playlists of what I judge to be the very best Go talks and tutorials available, here:

- [Bitfield Consulting on YouTube](#)

Acknowledgements

This book would not have been possible without Zack Proser, obviously (it would have sounded a bit weird if it were just me talking to myself).

It grew out of our many conversations about work, life, writing, and business, and our respective experiences of those things. After a while I said, “You know, this stuff is so interesting, it should really be a blog post.” Zack was kind enough to give me permission to make it so, but in the writing, it just grew and grew.

Each time we went over it together, our discussions about the text led to more questions, and those led to more answers. By the time it had expanded to a series of five blog posts, which got longer every time we revised them, I said “You know, this is starting to feel like it should be a book.”

And here it is. Thanks, Zack.